

(G), parent (P), and child (C) relationship. The figure also shows the Cartesian coordinates for each of the three points.

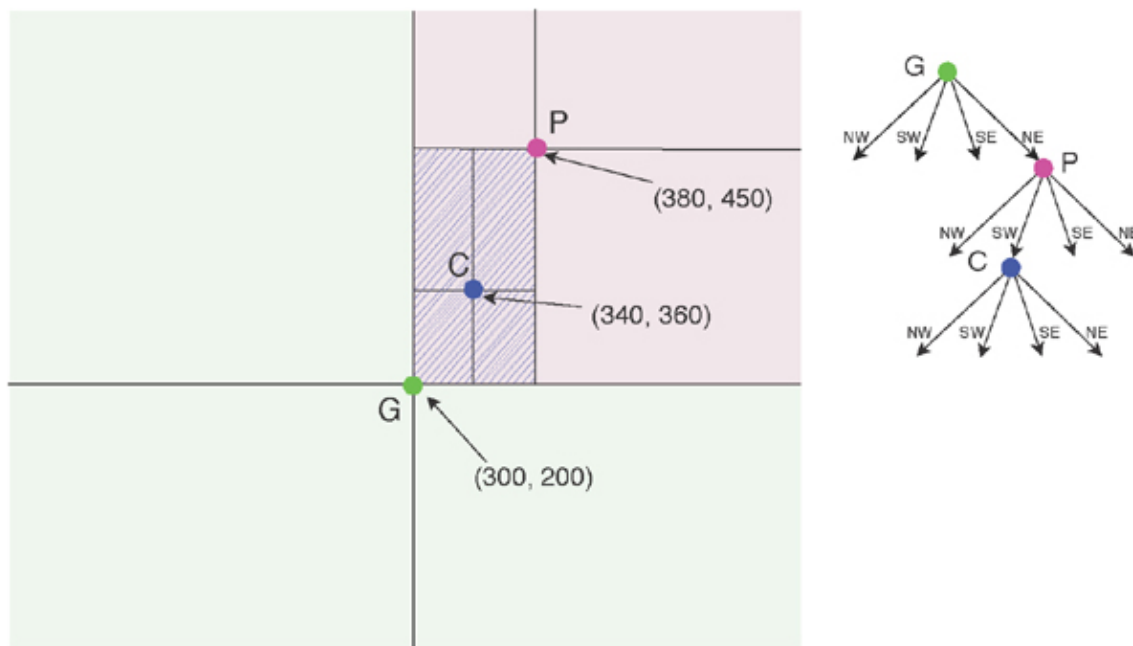


Figure 12-26 *The intersection of ancestor quadrants forms a child's quadrant*

What are the left, right, top, and bottom bounds of G's region of space? Because G is the root, its region of the plane is unbounded. In other words, its bounds are

left: `-math.inf`, right: `math.inf`, top: `math.inf`, bottom: `-math.inf`

When the search recursively descends to G's northeast child, node P, what are P's bounds? P's top and right bounds are still at infinity. However, P's left boundary is at G's *a* coordinate, 300, and P's bottom boundary is at G's *b* coordinate, 200. In other words, the bounds of P's region are

left: 300, right: `math.inf`, top: `math.inf`, bottom: 200

Taking one more step, what happens when the search recursively descends to P's southwest child, C. What are C's bounds (as shown in cross-hatched blue)? C's top boundary is defined by P's *b* coordinate 450, and C's right boundary is defined by P's *a* coordinate, 380. C's other boundaries are inherited from P. In other words, the bounds of C's region are

left: 300, right: 380, top: 450, bottom: 200

Each time you descend to a deeper node in the quadtree, the bounds for that child node are refined by replacing the appropriate two of the four bounds with the coordinates of the current node. In other words, the bounds of a node in the quadtree are formed by the intersection of the quadrant bounding boxes in the path from the root to that node.

Based on this understanding, take another look at the code for `__nearest()` in [Listing 12-28](#). For each of the four child quadrants, the method uses `Bounds.adjusted()` to create a new `Bounds` object, `newB`, appropriately updated to reflect the bounds of that child based on the bounds of its parent. That `newB` object determines whether the search circle's bounds intersect the child's quadrant's bounds.

Pulling It All Together: Implementing `QuadTree`'s `findNearest()`

As shown in [Listing 12-29](#), the public `findNearest()` method proceeds as follows. First, using the `__nearPoint()` method, it finds a candidate node, `ans`, by recursively descending from the root as far as possible into the quadtree. Then it computes the distance from the query point to the candidate node. This is the best answer so far, even though it's a rough approximation (unless the query point matches one of the points exactly). The candidate node and the distance from it to the query point define a query circle centered at the query point, with a radius equal to the distance to the initial candidate node.

Listing 12-29 *The `findNearest()` Method of `QuadTree`*

```
class QuadTree(object):
...
    # returns a,b,data,distance of point nearest to query a,b.
    def findNearest(self, a, b):
        if not self.__root: return None # No points yet!

        # Descend the tree and find the leaf node nearby
        # to this query point. This quickly sets an upper
        # bound on the nearest possible point
        ans = self.__nearPoint(self.__root, a, b)
        dist = self.__distance(a, b, ans.a, ans.b)
```

```

# Now we will descend the tree once more, refining
# the candidate by progressively shrinking the radius.
# The bounds of the root node are infinite.
bounds = Bounds()
ans, dist = self.__nearest(self.__root, a, b, dist, ans, bounds)

return ans.a, ans.b, ans.data, dist

```

Next, the `findNearest()` method constructs a `Bounds` object for the infinite extent of the root node. The root node, the query point, the current best candidate answer, and the infinite bounds are passed to the `__nearest()` method to begin the recursive search, which descends into any quadrant that is overlapped by the current query circle. Each time it encounters a `Node` that is closer to the query point than the current candidate, the candidate is replaced, and the radius of the query circle gets smaller. The result of the recursive call on the root node provides all the return values for the complete search.

The QuadTree Visualization tool makes it easier to see how the nodes are visited, how the distances change the query circle bounds, and how the bounds overlap. We show an example using the tool of a find nearest search starting, in [Figure 12-27](#). It begins by descending the 7-node quadtree to find the leaf node closest to the a, b query point. Starting at the root, P_0 , it goes to the southeast quadrant to visit P_2 . The query point lies in P_2 's northwest quadrant, so `__nearPoint()` descends to P_5 . P_5 is a leaf node, so returns it as the answer, `ans`.

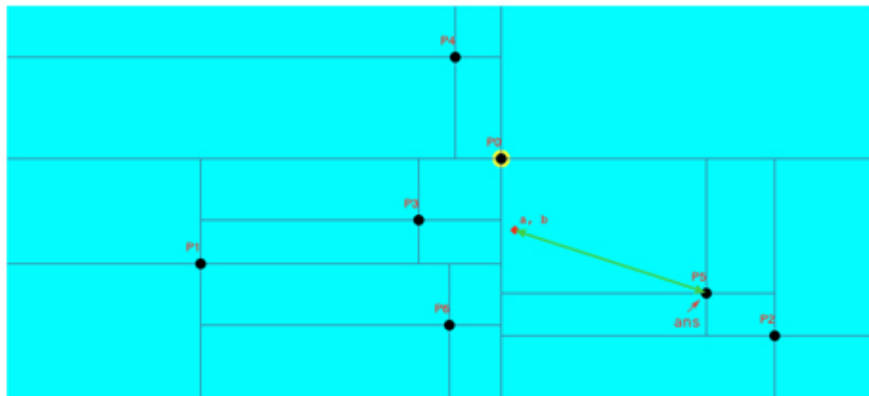


Figure 12-27 Finding the `nearPoint` during a find nearest search

Finding the near point and storing it in `ans` does not involve any distance calculations, only coordinate comparisons. The `findNearest()` method then computes the distance from the query point to the `ans`, as shown by the green line in [Figure 12-27](#). The resulting `dist` measure is passed to the first call to the `__nearest()` method as the query circle radius (the distance from the current `cand` to the query point).

The first call to `__nearest()` examines the root node, P0. After checking the base cases of visiting an empty leaf node or finding an exact coordinate match where `dist` is 0, the method measures the distance from P0 to the query point. [Figure 12-28](#) shows this measurement and makes it clear that P0 is closer than the candidate, `cand`, found by `__nearPoint()`, P5.

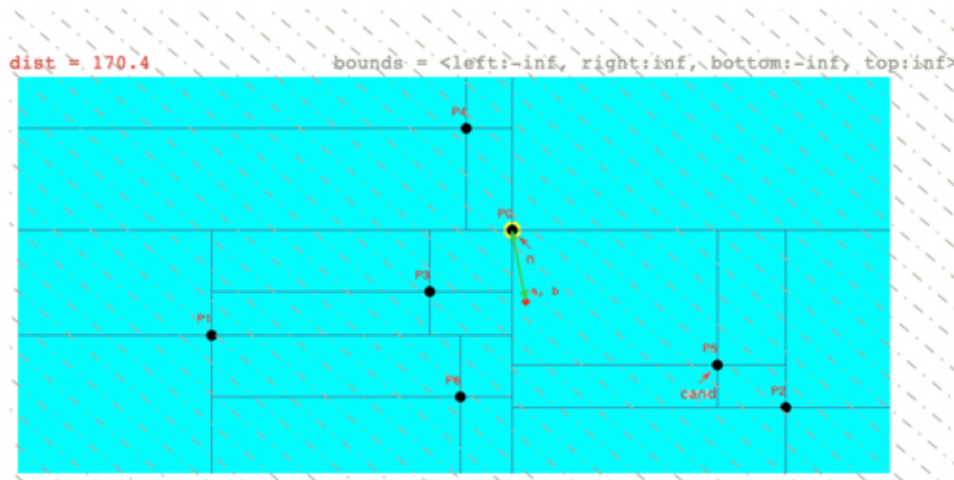


Figure 12-28 *Visiting the root node during a find nearest search*

The visualization tool also displays the various `Bounds` objects used by `__nearest()`. Initially, only the `bounds` value passed as an argument appears and has infinite extent. [Figure 12-28](#) shows the `bounds` value both at the top right as a string and with diagonal hashing in the same color over the entire display.

The distance from the query point (a, b) to P0 is 62.2, which is less than the `dist` measure to the `cand` point shown in the upper left. That means the `__nearest()` method replaces `cand` with the current node, P0, and updates the `dist` measure, reducing the query circle. That new radius is what's used to compute the circle bounds, `cBounds`, which is used to determine what quadrants to explore.

Figure 12-29 shows the situation when `__nearest()` is being called on the southeast child of the root. The `n` pointer indicates the current node, P2, while `cand` still points at P0. At this point, three `Bounds` objects are displayed (in addition to a faded version of the infinite bounds of the root node). The `bounds` region was passed from the calling routine and covers the whole southeast quadrant of P0. The `cBounds` region shows the query circle bounding box around *a*, *b*. That was made just large enough to include P0, the root and the current best `cand` point. It's shown in a different color and with a different angle to the hash marks. Finally, the `newB` region shows the region currently being checked to see whether it should be searched, P2's southwest quadrant. It's displayed in a third color and different hash angle. It's bounded on three sides by its point's coordinates and the bounds inherited from its parent, P0. Because `newB` doesn't overlap the `cBounds` region, it's clear that the southwest quadrant of P2 could not contain any points closer than current best `cand` point. Thus, the `__nearest()` method does not make a recursive call for the southwest quadrant of P2. It will, however, make a call for the northwest quadrant of P2 and visit node P5 because that quadrant does overlap the `cBounds` region.

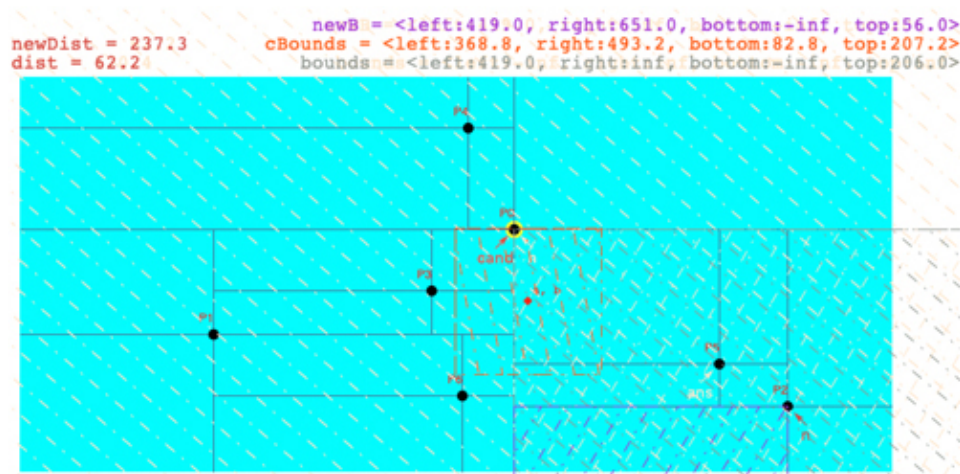


Figure 12-29 *Eliminating quadrants during find nearest search*

Although the overlapping information in the visualization tool may be confusing, the eliminated quadrants are somewhat easier to see. P2's SW, NE, and SE quadrants are eliminated, as are P5's NE and SE, P6's SW and SE, P1's NW and SW, P3's NW and SW, and P4's NW and NE quadrants. That's about half of the quadrants, which is similar to how binary search removes half of the range of cells to explore.

Efficiency of `findNearest()`

The time complexity of `findNearest()` is on average $O(\log N)$. The `__nearPoint()` method, which behaves similarly to `findExact()`, should be able to identify a candidate `Node` in $O(\log N)$ time, unless the tree is highly unbalanced. Likewise, a good initial candidate will result in a tight query circle, enabling `__nearest()` to efficiently descend back down into the tree considering just a small number of adjacent `QuadTree` cells, again in $O(\log N)$ time. An unbalanced tree or a degenerate distribution of data could result in $O(N)$ time for each search. For example, if all the `QuadTree` nodes are distributed in a circle, and the query point is at or very near the center of the circle. Then, `__nearest()` has no alternative but to visit every `Node` in the `QuadTree` because the search circle necessarily overlaps each nonempty `QuadTree` cell.

Theoretical Performance and Optimizations

`PointList`, `Grid`, and `QuadTree` present three different approaches to storing and querying spatial data points. [Table 12-1](#) summarizes the average Big O complexity of each operation implemented across spatial data structures.

Table 12-1 *Big O Performance of the Spatial Data Structures*

	PointList	Grid	QuadTree
<code>insert()</code>	$O(N)$	$O(N)$	$O(\log N)$
<code>findExact()</code>	$O(N)$	$O(N)$	$O(\log N)$
<code>findNearest()</code>	$O(N)$	$O(N)$	$O(\log N)$
<code>delete()</code>	$O(N)$	$O(N)$	$O(\log N)$
<code>traverse()</code>	$O(N)$	$O(N)$	$O(N)$

Could the Big O of `PointList` and `Grid` be improved to be competitive or superior to `QuadTree`?

One easy enhancement for `PointList` would be to use a hash table (such as a Python dictionary) instead of a list to store the entire collection of points in a `PointList`. Doing so would result in improved $O(1)$ performance for `insert()`, `findExact()`, and `delete()`. The performance of `findNearest()` and `traverse()`, however, would not be improved because both require considering every point in the `PointList`.

The `Grid` implementation already uses a hash table to map grid cell coordinates to lists of points stored in each grid cell. Because the grid spacing is fixed, however, the list in each grid cell contains $O(N)$ points, making the find and delete methods also $O(N)$. By replacing the list in each grid cell with a dictionary, the performance of `findExact()` and `delete()` can also be improved to $O(1)$. Here, too, `findNearest()` remains $O(N)$ because the nearest neighbor to query point is found by examining all the $O(N)$ points stored in one or more of the grid cells.

Even with these improvements, `QuadTree`'s theoretical $O(\log N)$ performance for `findNearest()` is difficult to surpass.

Practical Considerations

In this chapter we covered three different ways to organize spatial data: brute force using a list of points, uniform grids, and quadtrees. If you have dozens or even hundreds of spatial data points, then the brute-force approach may be sufficient.

But what should you use if you have many more points—say, tens of thousands, hundreds of thousands, or even millions? Understanding the Big O of a data structure provides some guidance. But remember that the Big O of an algorithm informs you only of the *shape* of the curve. It (intentionally) excludes an important constant factor that can make a difference in real-life applications.

Let's examine some practical time measurements of the performance of the `Grid` and `QuadTree` data structures. To do this, we ran experiments where we loaded points into a `Grid` or a `QuadTree` and then recorded the time to do 1,000 `findNearest()` queries. Geographic data points were randomly selected with a uniform distribution across a rectangle that roughly encompasses the continental United States. Then 1,000 uniformly distributed query points were

randomly selected across that same rectangle, and the time to find the nearest data point was recorded.

Figure 12-30 shows the results of these informative experiments. The X axis shows the number of points loaded into a `Grid` or `QuadTree` data structure. The Y axis shows the total time in seconds to run 1,000 `findNearest()` queries against the data. For grids, we created a `Grid` object where the cells were sized at 0.5 degrees in latitude and longitude—resulting in about 6,500 grid cells covering the United States. A second grid object was created where the cells were of size 1.0 degrees latitude and longitude—resulting in about 1,600 grid cells. Finally, we also created a `QuadTree`, also loaded with varying numbers of points, but able to grow in depth as more points were inserted.

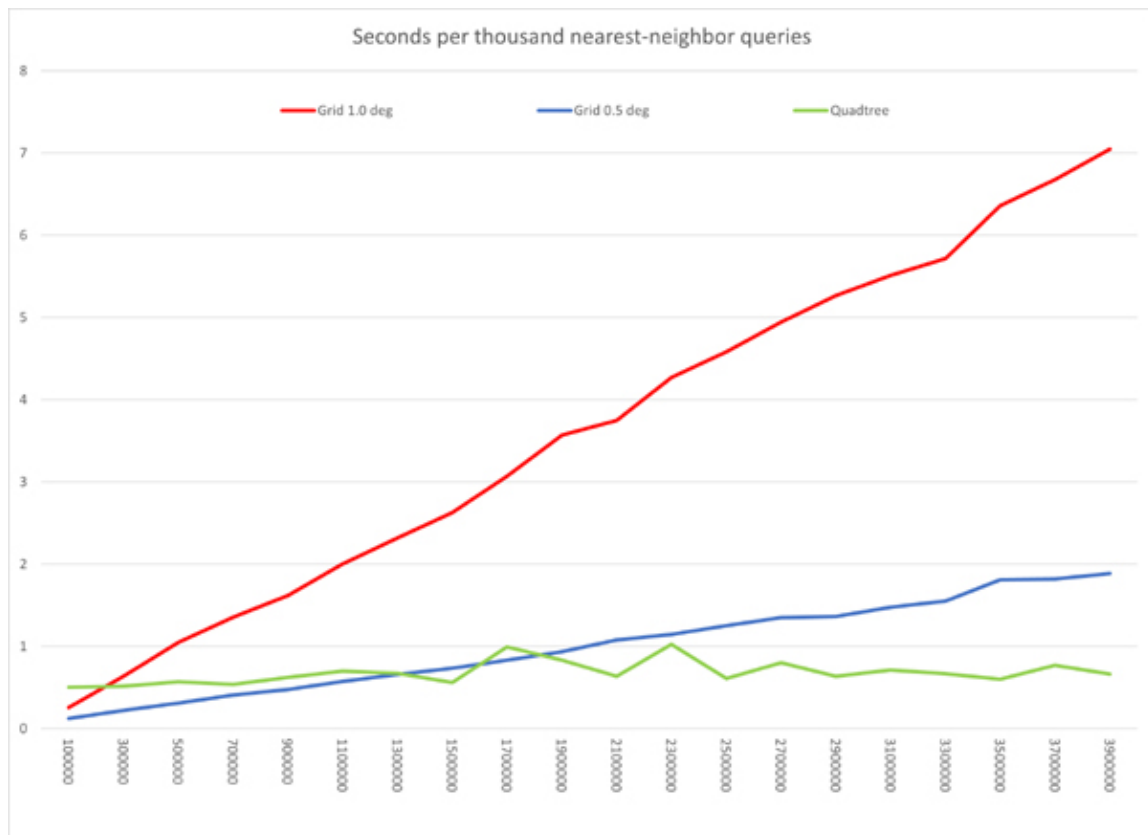


Figure 12-30 *Performance of `findNearest()` on different-sized data sets*

Not surprisingly, the red and blue plot lines are quite linear, confirming the analysis of their $O(N)$ performance. Their slopes reflect the respective sizes of the grid cells and, therefore, how many points are stored on average per cell. In the long run, as the data structures are loaded with millions of points,

QuadTrees win, with their very slow $O(\log N)$ growth as the number of points increases.

Interestingly, if you have only 100,000 or fewer points, then grids of cell size 1 or 0.5 degrees can perform better than quadtrees. If you were willing to expend more memory on grid cell storage, then an even finer grid would result in better linear performance, by reducing the leading constant factor. Be careful, however, in ensuring that the data won't grow much larger in software that uses grids like these.

These experiments were somewhat artificial because real-life data is usually not uniformly distributed across the map. Businesses, people, vehicles, and other real-life objects tend to be very clustered in space. If that's what your data looks like, then you should be especially careful to measure the performance of your spatial data structure under realistic conditions.

Further Extensions

We have explored the basics of spatial data: what it is, how you might organize points in a data structure, and how those organizations impact the four basic operations. The search operation, common to all data structures, needed extensions for exact matches and nearest match. Spatial data, however, is used for many different purposes, and those applications often require more capabilities than what we've described here. Let's explore a few extensions.

Other Operations

Beyond the `insert()`, `findExact()`, `findNearest()`, `delete()`, and `traverse()` operations we covered, there are many other possible—and useful—operations on spatial data. Here are just a few:

- Instead of finding just the one nearest neighbor to a point, find the K nearest neighbors to the query point.
- Traverse the points in order of increasing distance from a query point.
- Find all points within a specified distance from a query point.
- Find all points within a rectangular region of space.

- Find all points that lie within a polygonal boundary such as the borders of a country.
- Find the closest pair of points.

Each of these operations needs special algorithms to identify candidate points, determine whether they affect the output, and determine how to incorporate them in the output. Making them efficient is a challenge.

Higher Dimensions

The data structures covered in this chapter all deal with two-dimensional data—points on the plane or on the surface of the globe, specified with just two coordinates. Point data in three dimensions is also common. For example, a third dimension could correspond to the height of an object above the plane or globe. A natural extension to the `Grid` class could be used to store and query three-dimensional points. In effect, a three-dimensional grid would look like a regularly spaced collection of cubes (instead of squares).

Quadtrees also have a natural analog in the three-dimensional world, namely **octrees**. With quadtrees, each point recursively subdivides space into four quadrants. With octrees, each point subdivides space into eight *octants*. Analogous to map data, octrees are efficient at representing and searching three-dimensional point data that clusters in space.

It is possible to use spatial data structures to represent point data in even higher dimensions. You can use ***k*-d trees** (that is, *k*-dimensional) to represent points in arbitrarily high dimensions. These *k*-d trees are usually implemented by recursively subdividing space into half along carefully chosen dimensions.

These higher dimensional data structures can support the same types of operations, including insert, traverse, find exact, and find nearest.

Implementing and using data structures for high-dimensional data brings theoretical and engineering challenges. In a discussion of the distance between points in two-dimensional and three-dimensional space, you gain an intuitive understanding of what “distance” means. But when you consider additional dimensions, the notion of spatial distance is not meaningful and must be addressed carefully. For example, a fourth dimension of time could be very useful, but then what distance function would be useful in a space consisting of *x*, *y*, *z*, and time?

High-dimensional data also suffers from the “curse of dimensionality.” If you measure the Euclidean distance between two points of very high dimension, the differences in distances among many pairs of points can be very small. Another way to say this is that if you have high-dimensional data, all points in your data set can seem equally or nearly equally distant from each other! Therefore, although quadtrees and even k -d trees for low-dimensional data can be searched in $O(\log N)$ time, as the dimension of your data increases, it becomes necessary to search a larger and larger proportion of all the tree’s cells and data points to find a nearest neighbor!

Summary

- Spatial data structures store point data based on two-dimensional position.
- Positions can correspond to Cartesian x, y coordinates, or geographic latitude and longitude coordinates on the Earth.
- Distance between points is measured differently depending on the coordinate systems.
 - The Euclidean distance computes the straight-line distance between points on a plane.
 - The haversine distance computes the distance along the great circle connecting points on a globe.
- Point data can be stored in different kinds of spatial data structures.
- A brute-force approach stores points in a list or in an array such as the `PointList`.
- A `Grid` structure divides up the plane or globe into cells of equal size and stores point data as lists or arrays for each cell.
- Quadtrees organize point data as a tree.
- Each node in a quadtree stores data for a particular point in space.
- A quadtree node can have up to four child nodes corresponding to the four quadrants relative to its position.

- Grid structures do not adapt to point data that clusters spatially, but quadtrees do.
- All spatial data structures provide methods to insert new point data, search for an exact match, and search for the nearest point data to a query point.
- Grid searches begin in the query point grid cell and expand outward by layer.
- Quadtree searches begin at the root node and follow child quadrants containing the query point.
- Search performance is improved by eliminating grid cells or quadrants that lie outside of the current query circle.
- Query circles have a different shape when geographic coordinates are plotted with parallel lines for longitude meridians as in the Mercator projection.
- Rectangular bounding boxes simplify testing for the intersection or complete enclosure of one region and another but can only approximate the boundary of a query circle.
- Different spatial data structures yield different Big O performance.
- Simple point lists and grids take $O(N)$ time for search and deletion (and insertion when duplicate points must be detected and updated).
- Quadtrees take $O(\log N)$ time for insertion, search, and deletion.
- The performance of spatial data structures can be improved by augmenting the structure with $O(1)$ data structures such as hash tables.
- It is important to perform measurements with real-life data for your application to determine the best approach for your scale and type of data.
 - Uniformly distributed data can be organized well with grids, and the grid cells can be made small to improve performance.
 - Nonuniform data, or data of unknown size or distribution can be better organized with quadtrees because the quadtree adaptively subdivides

space in regions that contain more data.

- Octrees can store and search three-dimensional data.
- There are a wide variety of other operations needed for spatial data applications.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. What distinguishes the keys for spatial data structures from those used for other structures like trees, arrays, and lists?
2. _____ coordinates are used to represent a point on the surface of the earth.
3. When you're making data structures for two-dimensional spatial data, what things must be initialized when creating the (empty) structure for all of them?
4. Bounding boxes are used in spatial calculations because
 - a. rectangles are the only viable shapes that work with spatial coordinates.
 - b. checking whether two rectangles overlap is computationally simpler than checking whether an ellipsoid overlaps with a rectangle.
 - c. using bounding boxes is a convenient way to put a bound on the worst-case complexity of a spatial computation.
 - d. the haversine distance works only with rectangular bounds.
5. True or False: Lists of points are never an acceptable data structure for searching spatial data.
6. What advantage does a grid have over a point list in storing spatial data?
7. What advantage does a quadtree have over a grid in storing spatial data?
8. To implement efficient access to the cells of a grid, you use
 - a. a two-dimensional array or a list to store each cell of the grid containing data points.

- b. a quadtree to locate the quadrant storing the cell.
 - c. a list of point coordinates sorted by each point's row and column number.
 - d. a hash table that maps keys consisting of row, column tuples to the list of corresponding points in that grid cell.
9. Using Big O notation, how long does it take to find the nearest neighbor to a query point in each of the following structures holding N points?
- a. a point list
 - b. a grid
 - c. a quadtree
10. True or False: A quadtree supports data points with any number of dimensions.
11. Query circles
- a. are used in find exact searches to determine whether a point matches the query.
 - b. are used by quadtrees to bound the size of each node's quadrants.
 - c. are used by grids to limit which other grid cells should be traversed.
 - d. start large and decrease in radius as find nearest searches points in grids and quadtrees.
12. True or False: A quadtree uses layers to gradually expand the search for the nearest point.
13. What are the quadrants of a node? How are points lying on the boundary between two quadrants placed into the appropriate quadrant?
14. The depth of a well-balanced quadtree of N points is proportional to
- a. N
 - b. $N/4$
 - c. $\sqrt[4]{N}$
 - d. $\log_4 N$
15. True or False: Quadtrees are superior because their worst-case behavior is always better than point lists or grids.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

12-A Which of the three spatial data structures discussed in this chapter would use the least amount of memory per spatial data point? Which would use the most memory for the same number of points?

12-B Consider the problem of finding the item the farthest from a query point. How would you approach this problem, and which data structure would be more appropriate: a grid or a quadtree?

12-C The quadtree makes finding the nearest point very fast. Could the principles used in `findNearest()` be applied to finding the nearest key in other data structures? More specifically, how could you speed up the search for the nearest key in a binary search tree? Does the answer change if the keys are strings instead of integers?

12-C Consider the problem of finding the *K* nearest neighbors of a query point. How would you implement this method using your choice of spatial data structure, and what supplementary data structure might you use to make the implementation efficient?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

12.1 Modify the `Grid` class to support a `delete()` method that is given the coordinates of a point to delete. If the point is found in the `Grid`, then the point is deleted from the appropriate cell, and the associated data is returned to the client. If the point is not found, then `None` is returned to the client.

12.2 Modify the `QuadTree` class to support a `delete()` method. Because a `Node` object can't easily be removed from a `QuadTree`, consider adopting the representation that a `Node` with `None` for its data denotes a "deleted" node. When implementing `delete` this way, you must modify the class's

other methods to be consistent with this new representation. If the point is not found, then `None` is returned to the client.

- 12.3 Modify `Grid` and `QuadTree` to store more than one datum at a specific coordinate. In other words, change them to allow duplicate keys. Be sure to modify all methods that could be impacted by this change.
- 12.4 Create a subclass of `QuadTree` called `BulkQuadTree`. The new subclass has a constructor that accepts a sequence of points in bulk and then attempts to build a well-balanced quadtree. Add measurement methods to the quadtree to quantify its balance and measure the balance of quadtrees using large collections of random points and large collections of sorted points.
- 12.5 Add a method to `Grid` that moves the data associated with a point at a specified location to a new location.

13. Heaps

In This Chapter

- [Introduction to Heaps](#)
- [The Heap Visualization Tool](#)
- [Python Code for Heaps](#)
- [A Tree-Based Heap](#)
- [Heapsort](#)
- [Order Statistics](#)

Keeping items in priority order is useful in many contexts. At school and at work, we all deal with numerous tasks with various deadlines. Typically, the nearest deadlines get the highest priority. Sometimes, the importance of the task or the severity of not completing it outweighs a later deadline, so you move it up in priority and work it before tasks with shorter deadlines. When doctors and nurses triage patients arriving at a medical facility, they weigh decisions about the severity of the injury or illness and how long the patient can wait before treatment must start.

One characteristic of organizing tasks by priority is that you only need to know what the highest priority task is. That's the one that will be worked first. Perhaps you need to know what the first two or three tasks are because they can all be worked at the same time (or worked in smaller increments in rotation).

What's less obvious in prioritizing is that you don't need to know the precise order of the remaining tasks. You can set them aside and deal with them after higher-priority tasks are completed. That means you don't have to fully sort all the tasks; a partial sort allows you focus on the important ones.

You learned in [Chapter 4](#), “[Stacks and Queues](#),” that a priority queue is a data structure that offers convenient access to the data item with the smallest (or

largest) key. In those implementations, all the items are stored in a sequence sorted by their priority.

A **heap** organizes items in a *partially sorted order* inside a binary tree. That might seem as if it wouldn't be as useful as the binary search and other trees you saw in [Chapters 8, 9, and 10](#), but it is surprisingly powerful. The partial sorting means the heap is faster in some operations. That makes it particularly well suited for implementing priority queues, where it is much faster than maintaining a fully sorted sequence.

Heaps are used for more than just priority queues. They can be implemented with arrays, enabling an algorithm called a **heapsort** that competes with the quicksort both in speed and its low memory requirements. They are perfect for getting order statistics, such as the top 1 percent of a population of items when the rest won't be used (for example, finding the most important key words in a billion text documents from the Internet to summarize their content).

NOTE

Don't confuse the term *heap*, used here for a special kind of binary tree, with the same term used to mean the portion of computer memory available to store some kinds of data as the program runs. The computer memory heap is a section of random-access memory used to hold dynamically allocated and deallocated objects.

Introduction to Heaps

A **heap** is a binary tree with these characteristics:

- It's **complete**. This means all its levels are completely filled, except, perhaps, the leaf level. If you read the items from left to right across each row, either the whole row is full, or all the items are on the left side of the bottom row. [Figure 13-1](#) shows examples of complete and incomplete trees.

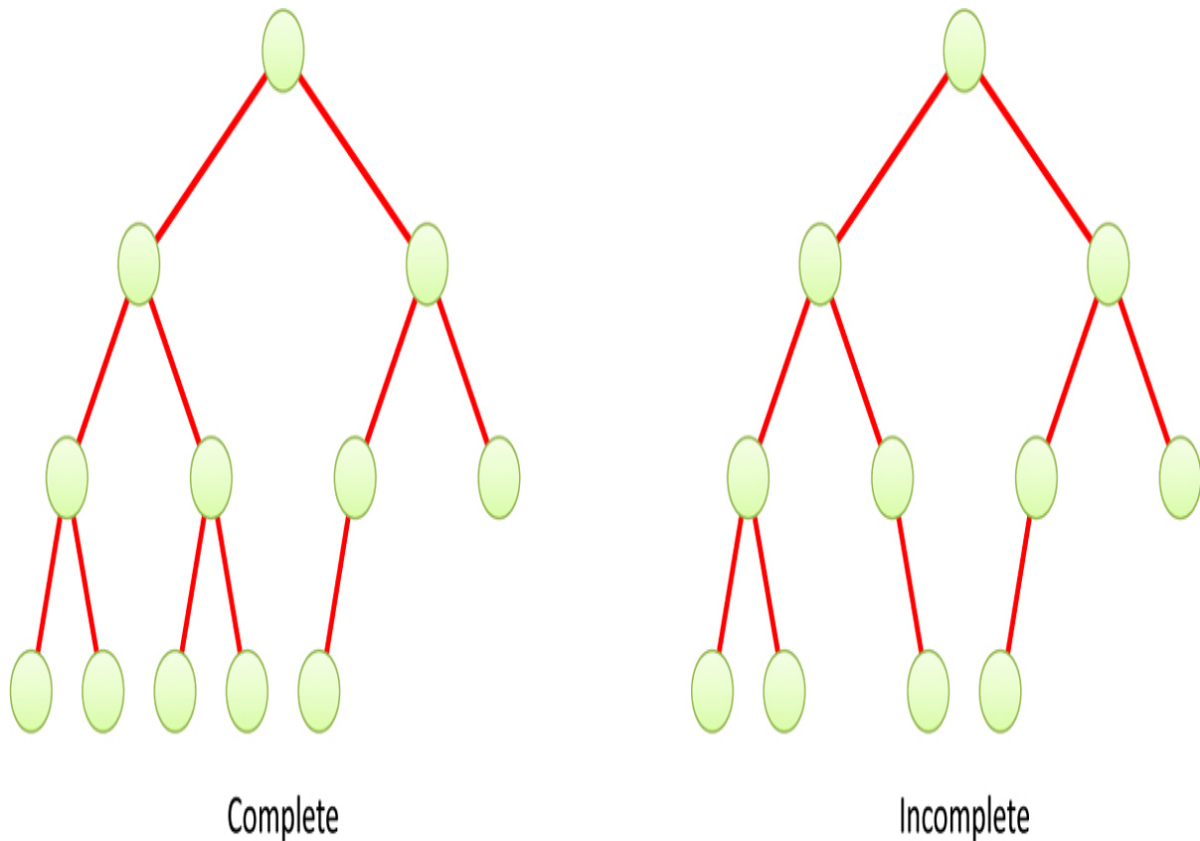


Figure 13-1 *Complete and incomplete binary trees*

- Each node in a heap satisfies the **heap condition**: every node's key is larger than (or equal to) the keys of its children. This makes the tree partially ordered.
- It's (usually) implemented as an array. This isn't a requirement, but it is a desirable feature in most use cases. In [Chapter 8, "Binary Trees,"](#) we described how binary trees can be stored in arrays, rather than using references to connect distinct node objects.

[Figure 13-2](#) shows a heap as both a tree and the array used to implement it. The array is what's stored in memory; the heap tree is only a conceptual representation. Notice that the tree is complete and that the heap condition is satisfied for all the nodes.

Another big difference from the other trees we've studied: the keys in the nodes are not in sorted order, neither in the array, nor in the binary tree, at least not the way they would be ordered in a binary search tree.

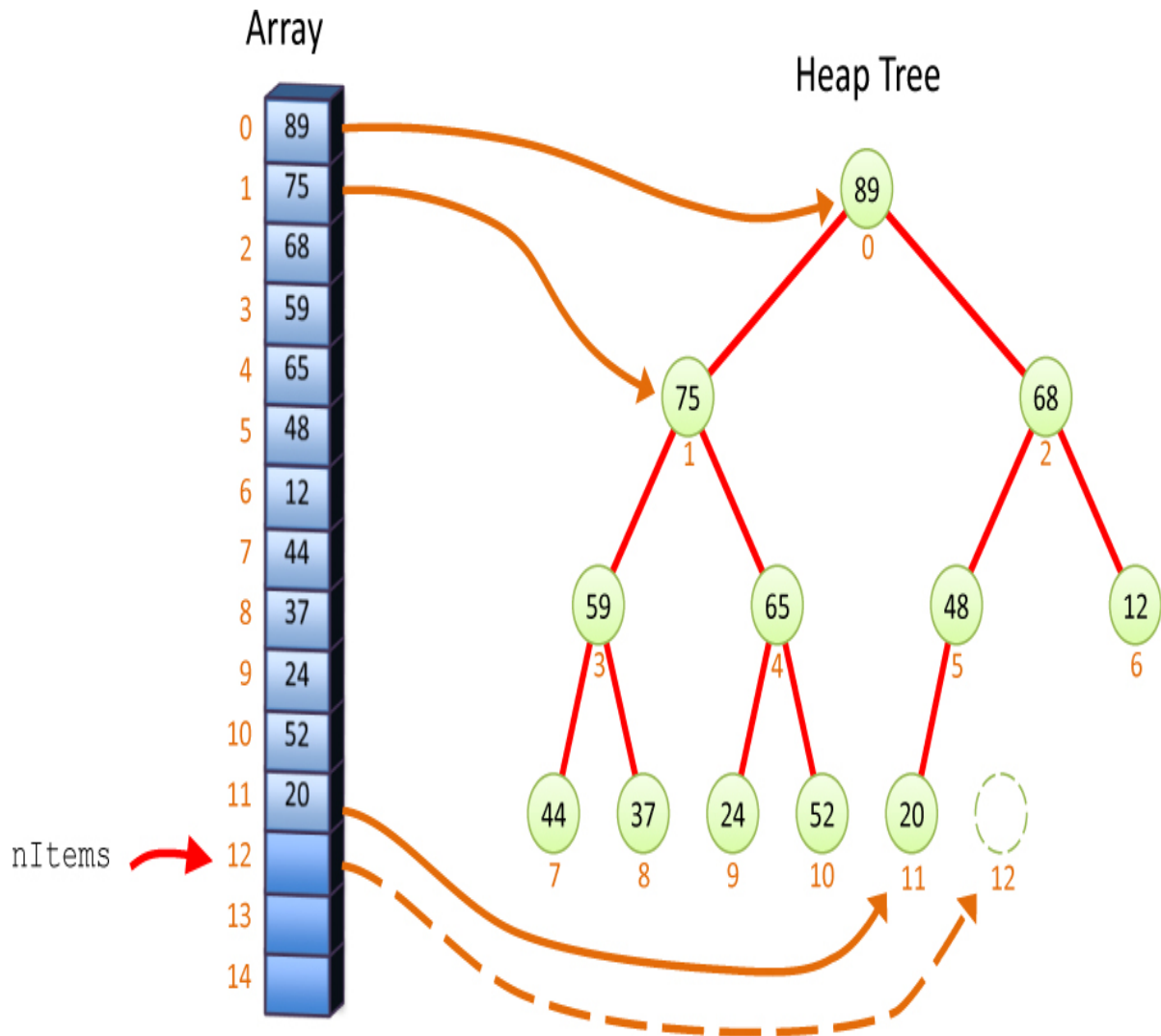


Figure 13-2 A heap array and its corresponding binary tree

The fact that a heap is a complete binary tree means there are no “holes” in the array used to represent it. Every cell is filled, from 0 to `nItems - 1` (`nItems` is 12 in [Figure 13-2](#)). When the next item is inserted, the cell at index `nItems` is filled, in the way that items were inserted at the end of arrays in [Chapter 2](#), “Arrays.”

We assume in this chapter that the maximum key (rather than the minimum) is stored in the root. A priority queue based on such a heap is a *descending-priority* queue. (We discussed ascending-priority queues in [Chapter 4](#).)

Priority Queues, Heaps, and ADTs

A priority queue is an abstract data type (ADT) offering methods that allow removal of the item with the maximum (or minimum) key value, insertion, and sometimes other operations such as traversal. We describe heaps in this chapter, which are frequently used to implement priority queues. There's a very close relationship between a priority queue and the heap used to implement it. The code fragment of [Listing 13-1](#) shows the equivalence.

Listing 13-1 *Implementing Priority Queues Using a Heap*

```
class Heap(object):
    def __init__(self, size=2): # Heap constructor
        self._arr = [None] * size # Heap stored as a list/array
...
    def insert(self, item): # Insert a new item in a heap
...
    def remove(self):      # Remove top item of heap and return it
...

class PriorityQueue(Heap): # Create a priority queue, using a heap
```

The three methods for the `PriorityQueue` class are identical to the methods for the underlying `Heap` class, so it can be implemented as a subclass. This example and the implementation in [Chapter 4](#) make it conceptually clear that a priority queue is an ADT that can be implemented in a variety of ways, whereas a heap is a more fundamental kind of data structure. In this chapter, for simplicity, we simply show the heap's methods without the priority-queue wrapping.

Partially Ordered

A heap is *partially ordered* compared with a binary search tree, where all a node's left descendants have keys less than all its right descendants. As you saw in [Chapter 8](#), the full ordering of binary search trees allows you to traverse the nodes in the order of their keys by following a simple algorithm.

In a heap, traversing the nodes in order is difficult because the organizing principle (the heap condition) is not as strong as the organizing principle in a binary search tree. All you can say about a heap is that, along every path from the root to a leaf, the nodes are arranged in descending order. As you can see in the tree of [Figure 13-2](#), the nodes to the left or right of a given node, or on higher or lower levels—provided they're not on the same path—can have keys

larger or smaller than the node's key. Except where they share the same nodes, paths are independent of each other.

Because heaps are partially ordered, some operations are difficult or impossible. Besides its failure to support traversal ordered by the keys, a heap also does not allow convenient searching for a specified key. The reason is that there's not enough information to decide which of a node's two children to pick in trying to descend to a lower level during the search. It follows that a node with a specified key can't be deleted, at least in $O(\log N)$ time, because there's no way to find it. These operations can be carried out by looking at every cell of the array in sequence, but this is only possible in slow, $O(N)$, time.

Thus, the organization of a heap may seem dangerously close to randomness. Nevertheless, the ordering is just sufficient to allow fast removal of the maximum node and fast insertion of new nodes. These operations are all that's needed to use a heap as a priority queue, as a sorting mechanism, and for finding certain members of a distribution. We discuss briefly how the two core operations are carried out and then see them in action in a visualization tool.

Insertion

Inserting an item in a heap is straightforward. You know it means adding a node to the tree, and the only place new nodes can go is at the leaf level. Because the tree is complete, the new node is added just to the right of the last node if the bottom row is not full, or as the left child of the leftmost node of a full bottom row. In [Figure 13-2](#), there are 12 nodes in the tree. Adding a thirteenth item means filling the array at index 12, which corresponds to the dashed node in the tree, just to the right of index 11.

What about the heap condition? If you're lucky, the inserted item has a key less than its parent (or it is the root node), and you are done. You can't count on being lucky, however, so you need to compare the keys. If the newly inserted leaf has a key greater than its parent, you can swap the two items. [Figure 13-3](#) shows an example where item 80 is inserted at the bottom in cell 12. The blue link indicates which two nodes need to be compared.

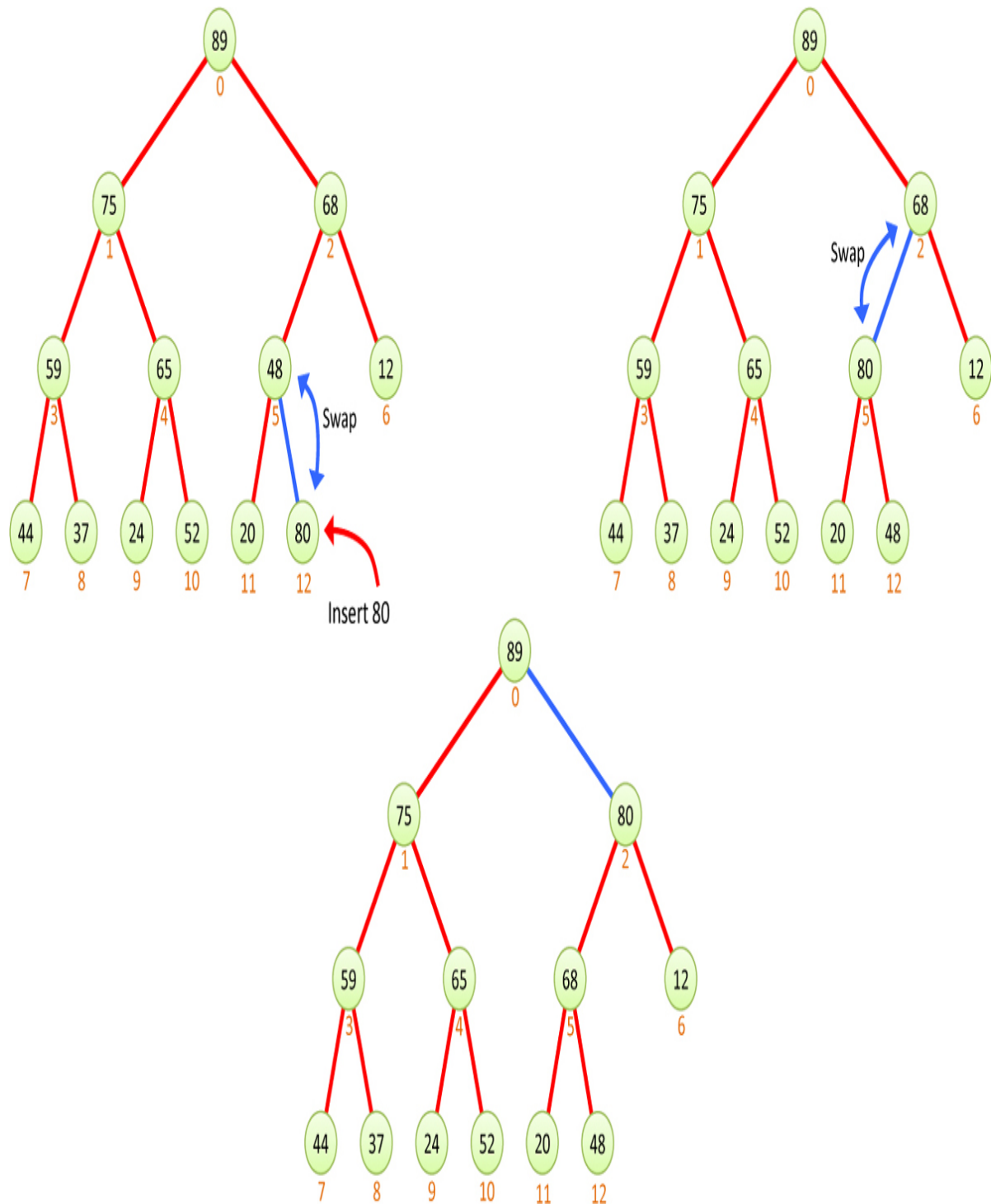


Figure 13-3 *Inserting an item in a heap and sifting up*

Because the inserted item's key, 80, is greater than 48, the key of its parent in cell 5, the items must be swapped. After the swap, item 80 now has items 20 and 48 as children, and both keys satisfy the heap condition because they are

smaller. Whenever you swap a parent and child key, you only increase the value of the parent key. That means that you can never break the heap condition with the other child—the one that wasn't swapped.

After moving the inserted item up to the second from bottom row, you still need to check whether it satisfies the heap condition in its new position. That's shown at the upper right of [Figure 13-3](#). Comparing 80 with 68 reveals that the two nodes linked by the blue line need to be swapped too.

After the second swap, item 80 lands in cell 2, as shown at the bottom of the figure. Comparing item 80 with its new parent, item 89 in cell 0, reveals that it now satisfies the heap condition. No more swapping is needed.

Moving items up the tree until they satisfy the heap condition is called **sifting up**. This operation is very similar in concept to one iteration of the bubble sort. The operation has also been called by other names such as *bubble up*, *trickle up*, *percolate up*, *swim up*, *heapify up*, and *up heap*. Because you apply it to a single new item on a heap that already satisfies the heap condition, only a single pass toward the root node is needed.

Removal

Removal means removing the item with the maximum key. In other data structures, like trees and hash tables, you use a `delete()` method to remove an item with a specified key. Heaps typically support the removal of the maximum key only. This item is always kept at the root node, so finding it is easy. The root is always at index 0 of the heap array.

The problem is that after the root is gone, the tree is no longer complete. Alternatively, from the point of view of the heap array, there's an empty cell. This "hole" must be filled.

There are several possible approaches to filling the hole. What if you simply shifted every item in the array by one cell toward index 0? That would get rid of the hole, but it could introduce new problems. If the array contained the three keys: 50, 30, and 40, deleting 50 would leave 30, 40. That could put 30 at the root with 40 as its one child, and that would break the heap condition.

What about using a modified version of the sift up method that was used for insertion? You could sift up the child of the (empty) root with the larger key. You could then repeat the process of filling the hole created by moving up the

child by moving down a level in the tree. You would continue until you move up a child node that is also a leaf node. That would certainly preserve the heap condition because you always move up the higher valued key. It doesn't appear to create any holes because it fills each vacancy it creates. Or does it?

In [Figure 13-4](#), two very similar heap trees are shown at the left. Removing the maximum item from the top, item 75, leaves a hole. The blue arrow shows the next step is to promote item 65 in cell 1 to fill the hole. The next step promotes item 44 to fill the hole at cell 1. At the end of the top row, the final heap has four items and the heap condition is preserved. So, the approach works for this heap.

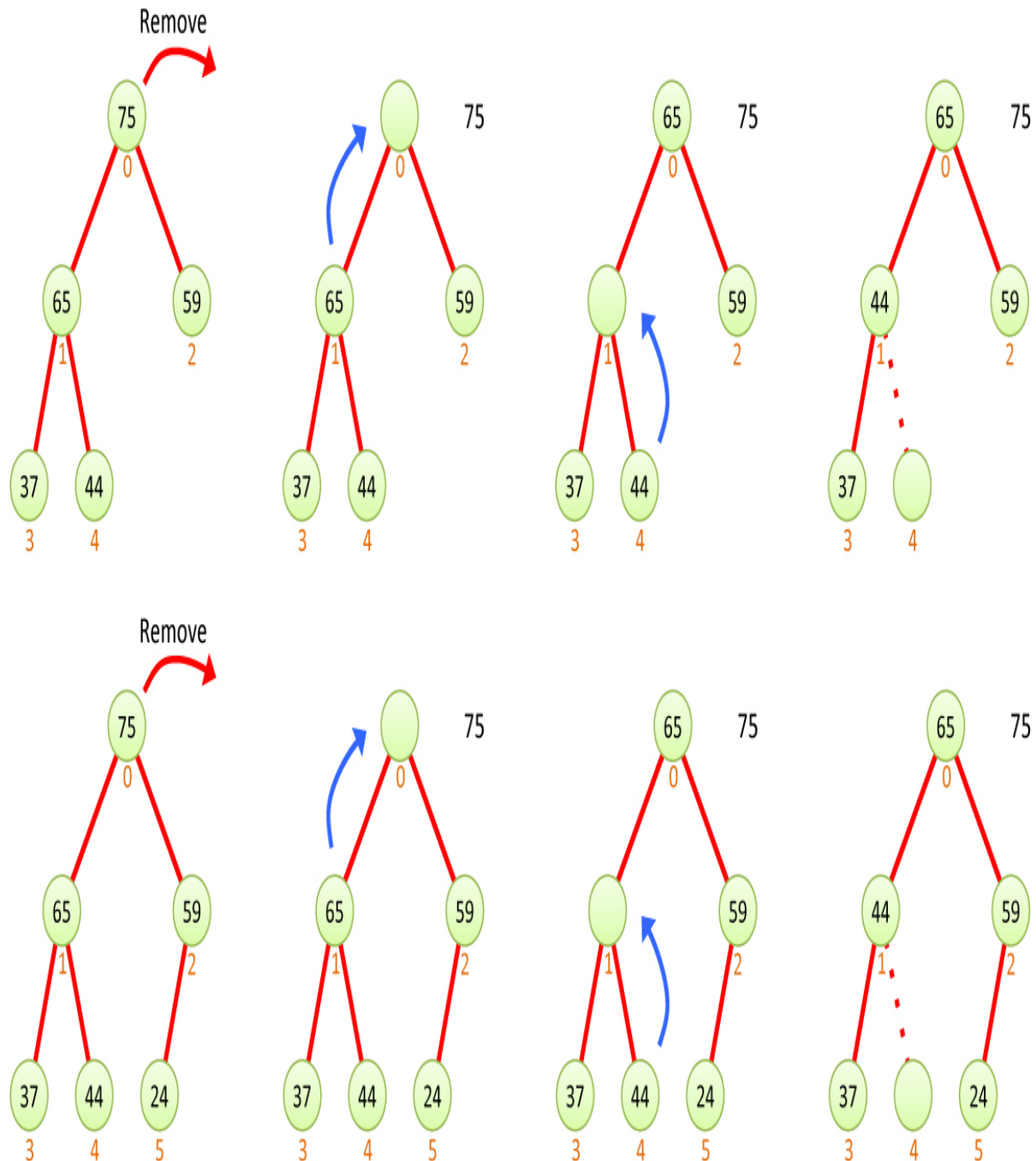


Figure 13-4 Two examples of filling holes with the maximum child item

In the bottom row of [Figure 13-4](#), however, the heap has one more item. The same items are moved in the same sequence. After you move up item 44, there is an empty cell 4 but a full cell 5, item 24. That empty leaf node means the tree is no longer complete and you don't have a heap.

Simply filling with the maximum child does not work. It seems as though you would need to sift up a child item based on the path to the last node in the tree; that is the node in the highest numbered filled cell. But if you sift up a child that has a key less than its sibling, it would create a node where the parent has a key less than its child. That would mean you would have to do more swaps or moves or actions like the rotations that were used to balance AVL trees. That's getting very complicated.

Fortunately, there's a simpler solution: take the root item out of the tree to return later, replace it with the last item in the heap, shrink the heap size by one, and **sift down** the new root. This operation is very similar to the approach of filling holes and solves the problem of leaving a hole in the wrong place on the leaf layer.

To see how this approach works, let's look at an example. [Figure 13-5](#) starts with the same initial heap as in the bottom row of [Figure 13-4](#). The first step is the removal of item 75 from the root node. The second step takes the last item in the heap, item 24 at cell 5, and moves it to the root. The third step is shrinking the heap size by one to eliminate cell 5.

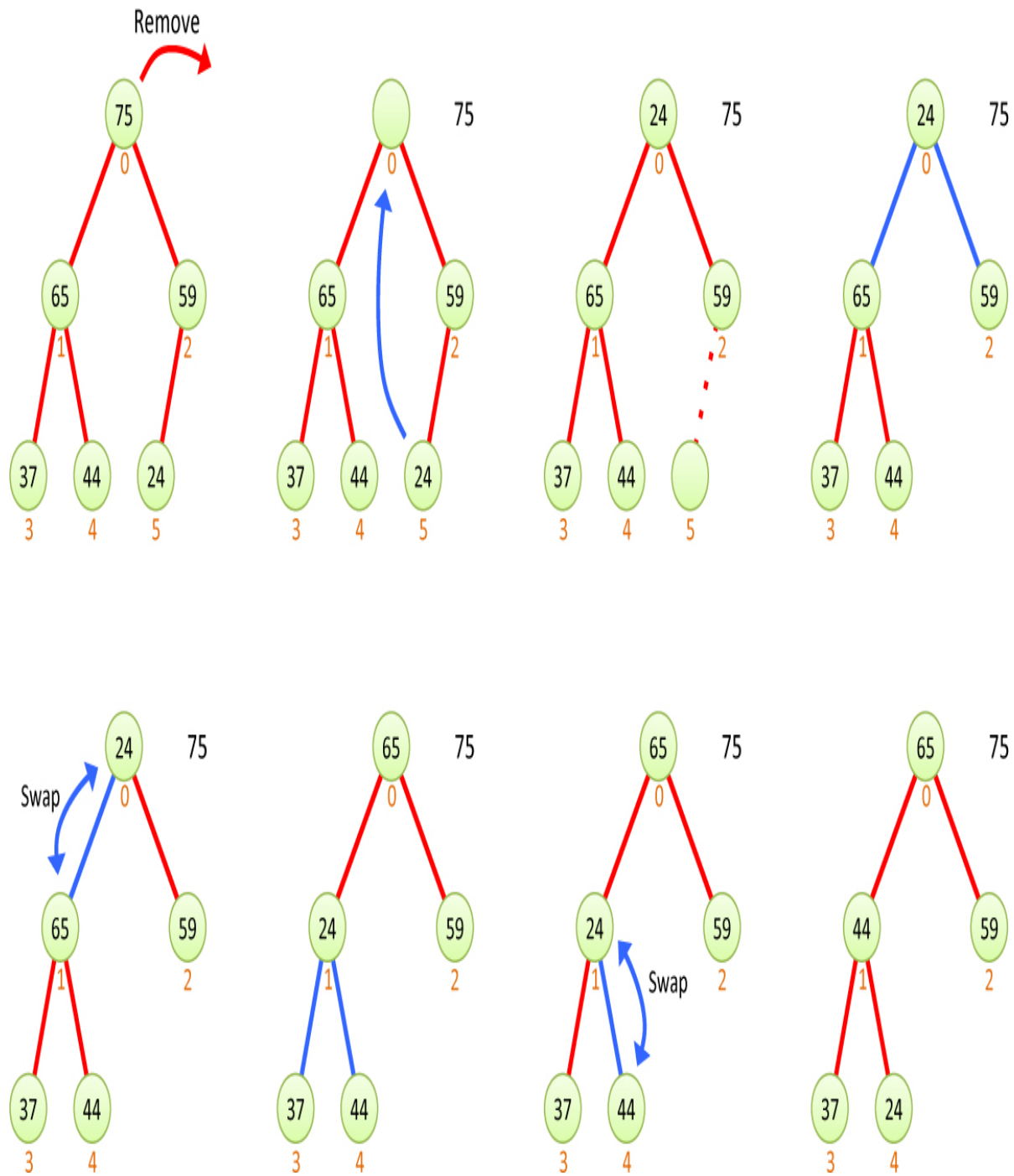


Figure 13-5 *Replacing the root with the last item and sifting down*

Now you can start the sift down process at the root node. In the upper right of the figure, item 24 sits at the root node. The blue links to its children indicate that we must look at their keys to determine which one is larger. The larger one (at cell 1) becomes the candidate for swapping.

In the next step—at the left of the second row in the figure—you compare the keys of cells 0 and 1 to find that a swap is needed. After swapping those two items, you begin the sift down process starting at cell 1. The two blue links below cell 1 show that you compare the keys of cells 3 and 4 to find which one is larger. After settling on cell 4, it's clear that another swap is needed. When that swap is made, you end up with the heap at the bottom right of the figure.

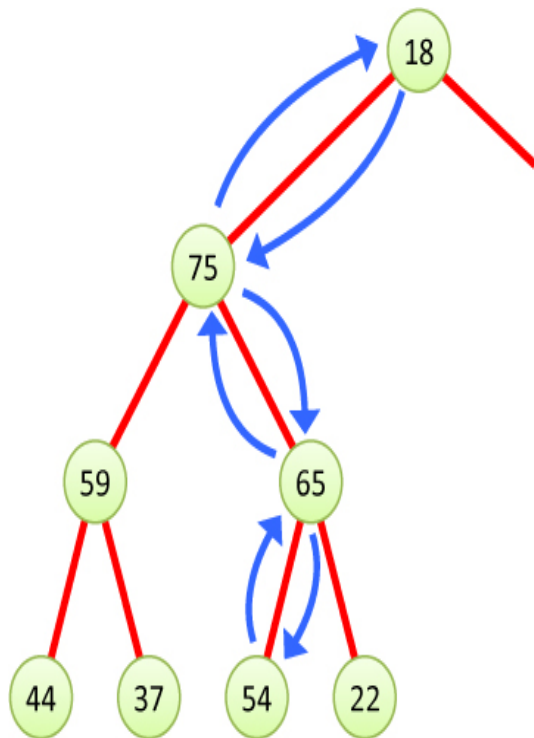
The final tree in [Figure 13-5](#) satisfies the heap condition and is complete. If you try to sift down further, you will find that cell 4 has no children, so no more sifting can be done. In fact, you can stop the process when either a leaf node is found, or no swap is needed with the maximum child.

As you might expect, this whole process is called *sift down* (or bubble down, trickle down, percolate down). It's only slightly more complex than sift up. The difference being that there are up to two children at each node. It must choose the node with the maximum key if there are two children, and the left node if there is only one. (There can never be a node with a right node and no left node in a complete tree.) Then it compares the parent with the target child, swaps them if needed and continues, or finishes if they are already in heap order. Because it always chooses the maximum child, the swap cannot create a heap condition violation.

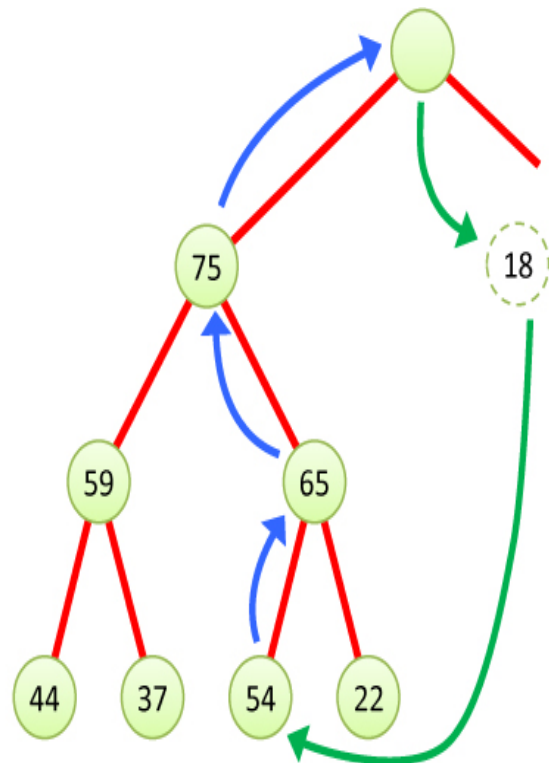
The sift down algorithm guarantees that the items in the nodes it visits are in heap order. It also guarantees the binary tree remains complete because removing the last node of a complete tree preserves completeness, and swapping parent and child nodes cannot create holes. Like sift up, this operation is similar to one iteration of a bubble sort. It also shares a characteristic with deletion in binary search trees where the successor node replaces the node to be deleted (without having to do rotations to rebalance the tree).

Not Really Swapped

Both the sift down and sift up algorithms are easy to understand when you visualize the item swaps between parent and child nodes like those shown in [Figure 13-3](#) and [Figure 13-5](#). Each swap involves two moves or copies, typically using a temporary variable. There's a more efficient way of implementing the changes, however, as shown in [Figure 13-6](#).



Sift down by swapping items



Sift down by moving items

Figure 13-6 *Sifting down with swaps versus with moves*

When you sift down, the item that was inserted at the root is moved down by each swap. Every item lower in the tree is compared with the same key of the item from the root. That means you can copy the root item into a temporary storage location and simply move items up into the holes that are created by each move/copy. The process is shown on the right of [Figure 13-6](#) and ends up being a rotation of the items.

Why is this process more efficient? In the example in [Figure 13-6](#), six items are moved when you use swapping. Five items are copied when you move items instead of swapping; three copies up (the blue arrows), plus one copy to the temporary variable and one copy from the temporary variable to the final position (the green arrows). That's not much of a savings, but think about sifting an item down 100 levels in a large heap. In that case, swapping makes 200 copies, whereas moving makes 102. You save about half the copy operations (and close to two-thirds considering a typical swap makes three copies using a temporary variable). The same method works when sifting up a newly inserted item at a leaf node.

Another way to visualize the sift up and sift down processes being carried out with copies is to think of a “hole” or “blank”—the absence of an item in a node—moving down in a sift down and up in a sift up. In [Figure 13-6](#), moving item 18 into the temporary storage at right leaves a hole at the root. When item 75 is copied up to the root, the hole moves down one level. The next move up of item 65 moves the hole down another level. Eventually, the hole moves to a leaf node or a node whose child keys are smaller than the item sifting down from the root. That hole is filled with the content of the temporary storage. The holes, of course, are conceptual; copying items in memory leaves the original value in the source node.

Sifting data down the heap saves time in the way the insertion sort saved time over the bubble sort. You use many fewer copy operations to achieve the result.

Other Operations

Heaps’ affinity to priority queues makes a few other operations very useful. In particular, priority queues used to manage the processes running on a computer can benefit by having methods for *peek*, *change-priority*, and *replace-max*. The peek operation is the same as for stacks and queues: it returns the key and data of the maximum keyed item without changing the heap contents.

The *change-priority* operation changes the priority of an item in the queue. This is needed when the priority of a process must be changed before it is next removed from the priority queue for running, perhaps because higher priority jobs are preventing it from being run.

The `change_priority()` method first must find the item in the heap, which could be quite slow, as mentioned in the “[Partially Ordered](#)” section. When it is found, increasing the priority means a sift up operation must follow. Similarly, decreasing the priority means sifting the item down. These changes can be performed just like insertion and removal, respectively, except that the sifting starts with an existing item anywhere in the tree after modifying its priority.

The *replace-max* operation is widely used. When operating systems take the next process item from the priority queue to run, they frequently have a process to put back in the queue to run later. The operating system usually limits the amount of time each process can run before it is suspended to allow other processes to run. The process that was running is placed back on the priority queue, perhaps with a different priority.

If you simply remove the maximum priority process/item from the heap and then insert the previously running process in the heap, there will be both a sift down for the remove operation followed by a sift up for the insert operation. A `replace_max()` operation eliminates the sift up operation by replacing the root item with the one to be inserted back in priority queue. The new root item is sifted down into proper position, and the old root is returned, so the `replace_max()` can be used in place of `remove()`. This approach saves considerable time in switching between hundreds or thousands of processes that might run for just a few microseconds.

The Heap Visualization Tool

The Heap Visualization tool demonstrates the operations we discussed in the preceding section: it allows you to insert new items into a heap and to remove or peek at the largest item. It does not, however, implement the other operations mentioned: change priority and replace max.

When you start up the Heap Visualization tool, you see a display like [Figure 13-7](#). The tool shows the heap both as an array and as a tree. Both are initially empty. The array is shown on the left side. It has two cells to start, and the `nItems` pointer points at cell 0 to show that no items are currently in the heap.

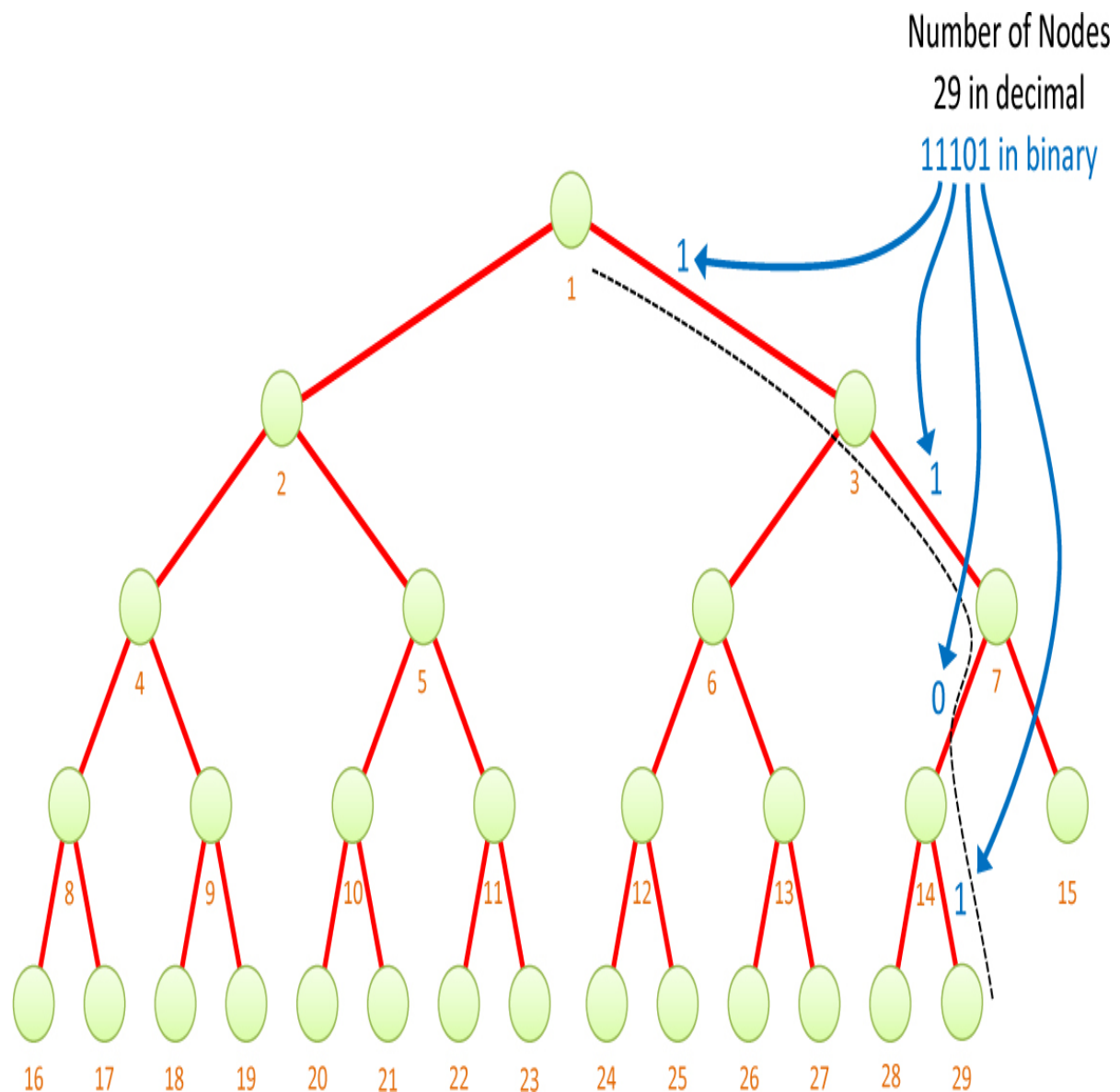


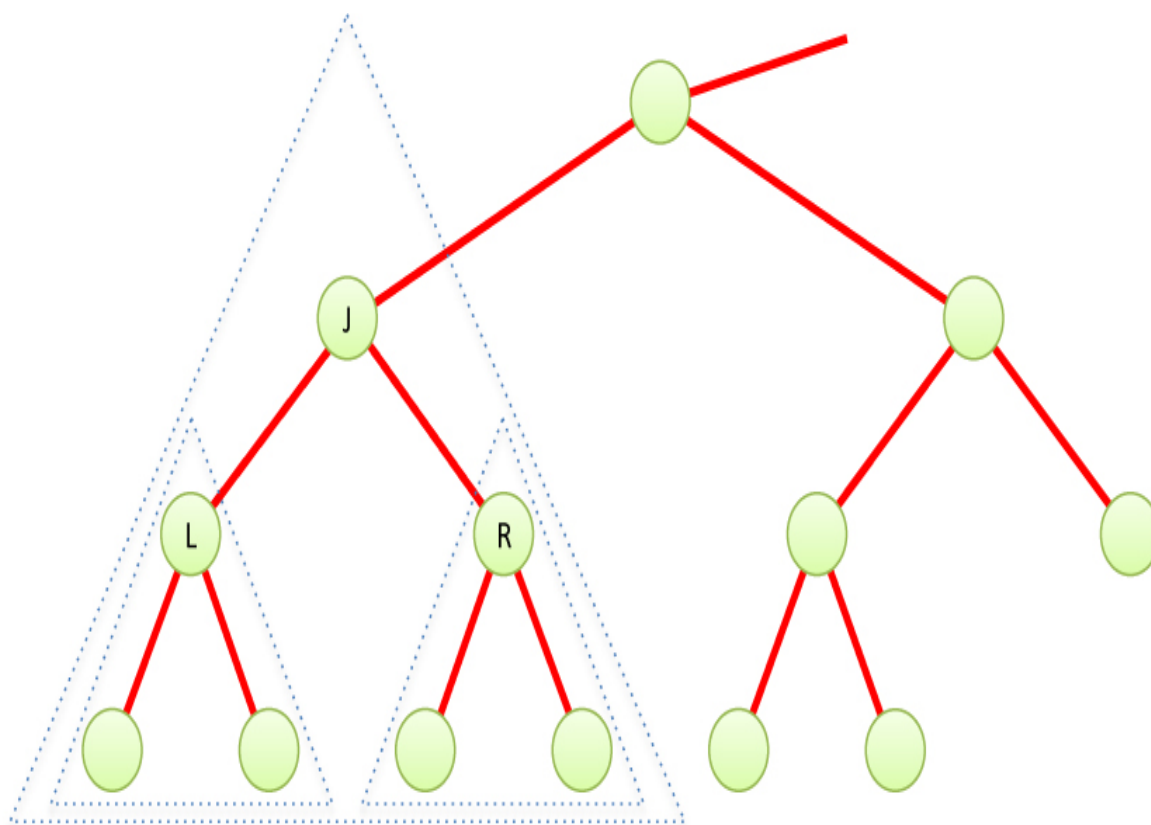
Figure 13-7 *The Heap Visualization tool*

The tree is shown to the right of the array. When the heap is empty, there are no nodes. Furthermore, because no tree object is needed when the tree is represented as an array, there's no box at the top like there was in the other tree visualizations.

The Insert Button

A new item is always inserted in the first available array cell—the one indicated by the `nItems` pointer. In the visualization, cell 0 is at the top of the

column of array cells to align with the tree that has its root at the top. The first node goes in cell 0 and creates a root node in the tree. Every subsequent insert goes in the array as a leaf node in the tree just to the right of the last node on the bottom row of the heap. Try entering a key—an integer between 0 and 99—in the text entry box and selecting Insert. If you enter the values 33, 51, 44, 12, and 70, you should see a heap like the one in [Figure 13-8](#).



If the subheaps at L and R are correct,
sifting down an item at J will create a correct subheap

Figure 13-8 *A small heap in the Heap Visualization tool*

During each insertion, you see the tool put the new item in the last array cell and last leaf of the tree, and then sift it up until it is correctly positioned according to the heap condition. You also see the array expand when needed, which we look at in the Python code.

The Make Random Heap Button

You can create heaps of up to 31 random items for experiments. Type the number of desired items in the text entry box and then select the Make Random Heap button. The array and the tree are filled with items, some of which may have duplicate keys. Duplicates are less of an issue in heaps because they aren't used like a database where items are identified by unique keys.

The Make Random Heap clears any existing items before it places the new items to satisfy the heap condition. If you wish to simply clear the current heap, you can enter 0 for the number of items to empty the heap.

The Erase and Random Fill Button

The Erase and Random Fill button performs a similar operation to that of the Make Random Heap button, except that it *only fills the array* with a randomly ordered collection of items. Because the heap condition is almost never satisfied after inserting items with random keys, the number of items in the heap is set to one (or zero if you choose an empty heap). A single-item array always satisfies the heap condition because there are no child items with which to compare its key.

The Visualization tool shows the random items in the array with the `nItems` pointer at cell 1, as shown in [Figure 13-9](#). With only one item satisfying the heap condition, only the root node is shown in the tree. If you insert a new item in this condition, the new item will go in the cell indicated by the `nItems` pointer, overwriting whatever item had been stored there earlier. A separate operation can be used to organize the array items beyond the `nItems` pointer into a proper heap.

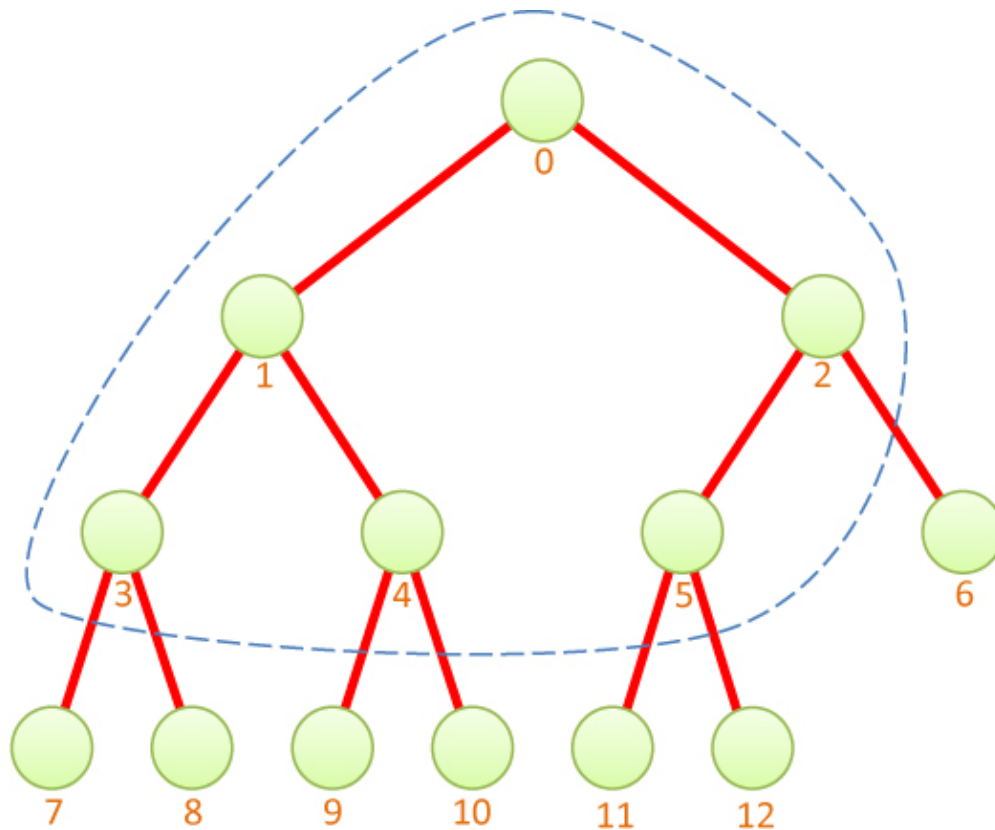


Figure 13-9 *A random set of items in the array used for the heap*

The Peek Button

Just as with stacks and queues, many algorithms need to see what the next item will be before they decide to remove it. The Peek button performs this common task, copying the key and data from the root node, the one with the maximum key, to return as an output.

The Remove Max Button

Selecting the Remove Max button takes the maximum item out of the heap, replaces it with the last item, and sifts that item down from the root of the tree. The removed key and data are returned as an output.

The Heapify Button

The Heapify button takes whatever data is stored in the array beyond the `nItems` pointer and rearranges it to form a heap. We study this operation in the “[Heapsort](#)” section later.

The Traverse Button

Occasionally, it’s useful to perform some operation on all the items in a heap, such as collecting statistics about the current collection. The Traverse button provides an example by printing the keys of all the items. We explore the issue of traversal order later in this chapter.

Python Code for Heaps

As mentioned previously, heaps are almost always implemented as arrays. The reason is that the corresponding binary tree is complete and always fills the first N cells of the array, where N is the number of items in the heap. We use Python’s `list` (array) to store the items in this implementation.

[Listing 13-2](#) shows the beginning of the definition of the `Heap` class. As with other array-based structures, the constructor takes an initial `size` parameter to determine the size of the array to allocate. An internal `_nItems` field is set to 0 to indicate no items are stored in the private `_arr` yet. The constructor also takes a `key` parameter, which defines the function to extract the key used in ordering records or other complex structures stored in the array.

Listing 13-2. *The Core Heap Class*

```
def identity(x): return x    # Identity function

class Heap(object):
    def __init__(self, key=identity, size=2): # Heap constructor
        self._arr = [None] * size # Heap stored as a list/array
        self._nItems = 0         # No items in initial heap
        self._key = key          # Function to get key from heap item

    def isEmpty(self): return self._nItems == 0 # Test for empty heap

    def isFull(self): return self._nItems == len(self._arr)

    def __len__(self): return self._nItems    # Number of items
```

```
def peek(self):          # Return item with maximum key
    return None if self.isEmpty() else self._arr[0]
```

The first four methods are like those for stacks and queues. The `isEmpty()` method tests whether the heap has any items stored in it. The `isFull()` method checks whether the number of items fills the entire array that was allocated. The `__len__()` method allows callers to know the number of items in the heap using Python's built-in `len()` function. Finally, the `peek()` method returns the maximum-keyed item in the heap if it is not empty.

The next methods define the correspondence between the array and tree representations of a heap. Traversing a tree when sifting up or sifting down means following parent-child relationships, which are not explicit in the array. As you saw in [Chapter 8](#), the node at index `i` in the array has its

- Parent at $(i - 1) // 2$,
- Left child at $i * 2 + 1$, and
- Right child at $i * 2 + 2$.

Those relationships can be seen in [Figure 13-2](#). The formulas may return indices that are outside of the array bounds, $[0, _nItems)$, in which case there is either no parent or no child for that node. The methods in [Listing 13-3](#) translate cell indices into those of their neighboring nodes but do not test whether there is an item there.

Listing 13-3 *Parent and Child Relationship Methods*

```
class Heap(object):
...
    def parent(self, i):          # Get index of parent in heap tree
        return (i - 1) // 2      # Item i's parent index is half of i - 1

    def leftChild(self, i):       # Get index of left child in heap tree
        return i * 2 + 1         # Item i's left child is at twice i plus 1

    def rightChild(self, i):      # Get index of right child in heap tree
        return i * 2 + 2         # Item i's right child -> twice i plus 2
```

NOTE

Remember that Python's `//` operator performs integer division, in which the answer is rounded to the lowest integer. This is true even for negative numbers: `-1 // 2 == -1`.

Insertion

The `insert()` method is fairly short; the bulk of the work is handled by the sift up algorithm in a separate method, as shown in [Listing 13-4](#).

If the internal array is full, the `insert()` method calls the private `_growHeap()` method to increase the size of the array. Let's look at growing the heap before the rest of the insert operation. Like the hash tables in [Chapter 11](#), "Hash Tables," growing the heap doubles the size of the array. First, it saves the current array in a temporary variable, allocates a new array that's twice as large (including at least one cell), and then copies all the current items into it. Unlike the hash tables, there is no need to reinsert (rehash) the items in the heap; they can be copied to the new array in the same order. The Visualization tool animates the `_growHeap()` process, and seeing it in operation can help clarify the details.

After the array size has been checked and perhaps enlarged, the `insert()` method places the new item in the empty cell indexed by `_nItems`. After incrementing `_nItems` by one, it can now call `_siftUp()` on the new item at the end of the array (bottom of the tree).

The `_siftUp()` method starts at a particular index in the array, `i`, and works up the parent links to the root. If called on the root (or a negative index), it does nothing because the item cannot move upward in the tree. Otherwise, it prepares to move the `item` at `i` by storing it and its key in temporary variables. This creates the first "hole" at cell `i`.

The `while` loop "walks" up the parent links. When `i` gets to 0, it has reached the root, and no more moves are possible. In the loop body, `_siftUp()` finds the parent node and compares its key to that of the item being sifted up. If the parent's key is less than the `itemkey`, it moves the parent item down into cell `i`. That means the "hole" moves up, and it changes `i` to point at the parent cell. If the parent's key is greater than or equal to the `itemkey`, then the final position for the item has been found and the loop terminates.

The final step is moving the `item` to the "hole," where it either has a parent with a larger key or no parent node. The hole is now filled, and the heap condition is restored.

Listing 13-4 *The insert() Method for Heaps*

```
class Heap(object):
...
    def insert(self, item): # Insert a new item in a heap
        if self.isFull(): # If insertion would go beyond array
            self._growHeap() # then expand heap array
        self._arr[self._nItems] = item # Store item at end of array
        self._nItems += 1 # Increase item count
        self._siftUp(self._nItems - 1) # Sift last item up

    def _growHeap(self): # Grow the array for the heap
        current = self._arr # Store the current array
        self._arr = [None] * max(1, 2 * len(self._arr)) # Double array
        for i in range(self._nItems): # Loop over all current items &
            self._arr[i] = current[i] # copy them to the new array

    def _siftUp(self, i): # Sift item i up toward root to preserve
        if i <= 0: # heap condition. The root node, i = 0,
            return # cannot go higher, so done.
        item = self._arr[i] # Store item at cell i
        itemkey = self._key(item) # and its key
        while 0 < i: # While i is below the root
            parent = self.parent(i) # Get the index of its parent node
            if (self._key(self._arr[parent]) < # If parent's key is
                itemkey): # less than that of item i,
                self._arr[i] = self._arr[parent] # copy parent to i
                i = parent # and continue up tree
            else: # If parent's key is greater or equal,
                break # then we have found where item i belongs

        self._arr[i] = item # Move item i into final position
```

Removal

The removal algorithm is also not complicated. As with insertion, the method has two parts: 1) the initial work of removing the maximum keyed item and filling that hole, and 2) sifting down the new root item. [Listing 13-5](#) shows the code. The first step is to raise an exception if the heap is empty. Next, it copies the item in the root node to a temporary variable. This original `root` is returned after reducing the heap.